

NLP Assignment 2 — Vectors & Language Modelling

Author: Tauha Imran

Reg. No: 22i1239 (G4)

Course: Natural Language Processing (NLP)

Assignment: A2 — Vectors & Language Modelling

Abstract

This report presents the implementation and results of an NLP pipeline that processes legal documents using optical character recognition (OCR), tokenization, and neural embedding models. The goal was to build a compact system for summarizing Pakistan Supreme Court judgments through a combination of extractive and abstractive techniques. The system leverages both custom-trained embeddings and TF-IDF-based fallback mechanisms to ensure robustness and full summary coverage across the dataset.

Introduction

Natural Language Processing (NLP) involves transforming unstructured text into structured, machine-understandable representations. This assignment focuses on text vectorization, a fundamental step in understanding textual semantics. The task involves extracting text from scanned legal documents, preprocessing it, generating embeddings, and performing automatic summarization. The work is divided into multiple parts covering OCR, tokenization, model training, and summarization.

The notebook was developed and tested locally using Python 3.8+ and libraries such as NumPy, NLTK, scikit-learn, and spaCy. It uses both extractive and abstractive summarization approaches to generate concise yet meaningful summaries of long legal texts.

Methodology

1. OCR and Preprocessing

- **Libraries:** `pdf2image`, `pytesseract`, and `Pillow` were used to convert PDF pages to images and extract text.
- The OCR module reads scanned court judgments and outputs plain text.
- Text cleaning involved removing noise, special characters, and redundant whitespace.

2. Tokenization

- Sentence and word tokenization were implemented using **NLTK's Punkt** tokenizer.
- Tokenized sentences formed the basis for subsequent vectorization and embedding models.

3. Word Embedding Model

- Implemented a **skip-gram style neural network** using **NumPy** to learn word embeddings from scratch.
- Each word was mapped to a fixed-dimensional vector capturing semantic relationships.
- Embeddings were later averaged to obtain **sentence-level representations**.

4. Sentence Embeddings and Summarization

- **Extractive Summarization:** Based on sentence embeddings; sentences closest to the centroid were selected as summary candidates.
- **Abstractive Summarization:** Rule-based reformulation of extractive outputs to make summaries more fluent.
- **TF-IDF Fallback:** If embeddings failed or mismatched, a TF-IDF-based summarizer ensured non-empty summaries.

5. Diagnostics and Repair

- Diagnostic cells helped verify and repair empty or malformed summaries.

- Repaired outputs were written to `outputs/final_summaries_fixed.jsonl` to maintain data integrity.
-

Results

- The OCR successfully processed all input judgments, converting them into usable text files.
- The skip-gram model produced meaningful word and sentence embeddings, enabling effective extractive summarization.
- The hybrid summarization pipeline achieved **robust coverage**, ensuring all documents produced summaries even in cases of missing embeddings.
- Final results were stored as JSON Lines (`.jsonl`) files for easy grading and inspection.

Example output file:

`outputs/nlm_results_complete.jsonl`

Each line contains a JSON object with the document ID and its corresponding summary.

Discussion

The combination of extractive and abstractive summarization ensures high reliability. While abstractive summaries tend to be more fluent, extractive summaries guarantee factual accuracy. The inclusion of a TF-IDF fallback provides resilience against OCR noise and embedding mismatches.

Manual checks confirmed that summaries retained the main themes and legal conclusions of the original judgments, even in low-quality OCR cases.

Conclusion

This assignment demonstrates a complete NLP pipeline—from OCR to summarization—built primarily with Python and classical NLP libraries. Despite operating without large pretrained models, the system achieves coherent and robust summarization of lengthy legal documents. Future work may involve replacing the NumPy-based skip-gram with transformer embeddings (e.g., BERT) for enhanced contextual understanding.

Environment & Dependencies

- **Python:** 3.8+
 - **Libraries:** numpy, nltk, scikit-learn, spacy, pdf2image, pytesseract, Pillow
 - **SpaCy Model:** `en_core_web_sm`
 - **Platform:** Tested on CPU; GPU optional
-

References

1. Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," 2013.
 2. NLTK Documentation — <https://www.nltk.org>
 3. spaCy Models — <https://spacy.io/models>
 4. Scikit-learn TF-IDF Vectorizer — <https://scikit-learn.org>
-

End of Report